

Real-Time Notification System

1. NotificationService Architecture & Channel Workers

When `NotificationService` consumes an event from Kafka topics, it inspects recipient notification preferences and dispatches tasks to dedicated, isolated worker pools:

- **Email Worker:** Assembles rich HTML tickets containing QR codes and booking summaries, dispatched via email service providers (AWS SES / SendGrid).
- **SMS Worker:** Delivers concise transactional messages containing booking codes and screen details via telecom gateways (Twilio / AWS SNS).
- **Push Notification Worker:** Delivers immediate mobile alerts to iOS and Android devices via Firebase Cloud Messaging (FCM) and Apple Push Notification service (APNs).
- **WebSocket Gateway:** Broadcasts real-time UI state updates directly to active browser and app sessions, updating the user's screen to "Booking Confirmed" without manual page refreshes.

2. Delivery Guarantee: At-Least-Once with Consumer Idempotency

We implement **At-Least-Once delivery combined with Idempotent Consumer Processing**. Because distributed network interruptions can trigger Kafka message re-delivery, `NotificationService` checks Redis for the unique `eventId` before invoking any delivery gateway:

```
Algorithm: ProcessNotificationEvent(event)
1. If redis.exists("event:processed:" + event.eventId):
    Return
2. Resolve user delivery preferences and contact details
3. Dispatch notification to active channel workers
4. redis.setex("event:processed:" + event.eventId, 86400, "COMPLETED")
```

If a duplicate event arrives within a 24-hour window, the service acknowledges the message offset immediately and skips execution, guaranteeing that users never receive duplicate SMS or email tickets.

3. Retries, Exponential Backoff, and Dead-Letter Queue (DLQ)

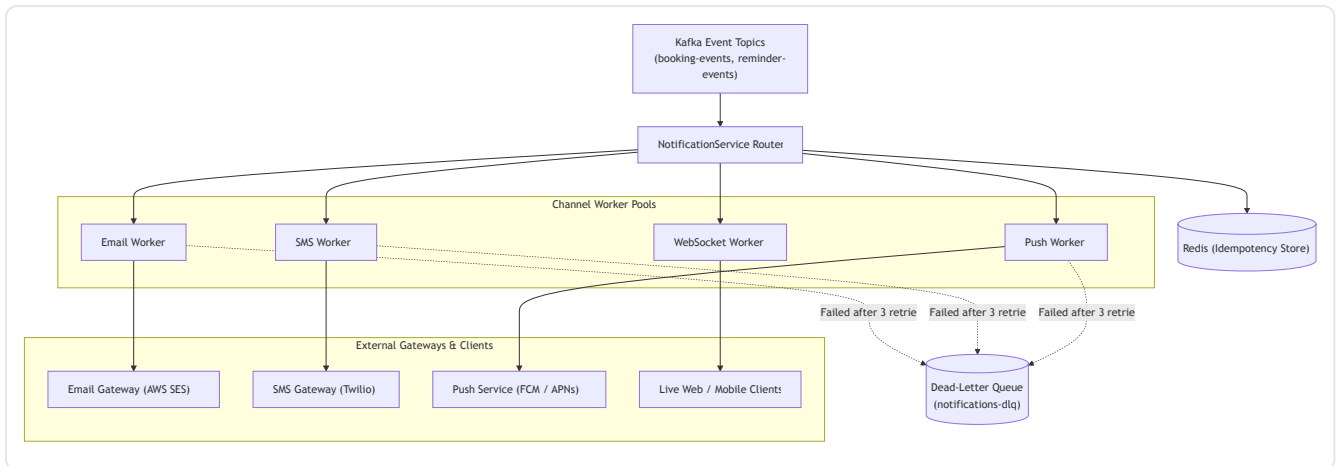
Third-party communication providers periodically experience rate limits, gateway timeouts, or intermittent downtime:

- **Exponential Backoff:** If an external gateway call fails, the worker retries up to 3 times with exponential backoff (1s, 2s, 4s).
- **Dead-Letter Queue (notifications-dlq):** If all 3 retry attempts fail, the message is routed to the dead-letter topic `notifications-dlq` along with the failure reason and timestamp.
- **Partition Non-Blocking:** Once the message is routed to the DLQ, the consumer commits the Kafka offset on the primary topic, allowing subsequent customer notifications to process without delay.

```
Algorithm: DeliverWithRetry(channel, payload)
1. attempt = 0, max_retries = 3
2. While attempt < max_retries:
    try:
        result = channel.Send(payload)
        Return result
    catch GatewayException:
        attempt = attempt + 1
        Sleep(2 ^ attempt seconds)
3. PublishToDLQ("notifications-dlq", payload)
```

4. Component Diagram: Multi-Channel Delivery

The component diagram below illustrates how incoming events are validated, routed to channel workers, and delivered to external gateways:



5. Worker Scaling & Rate Limiting Strategy

- **Independent Worker Scaling:** Each notification channel operates as an independent worker pool. This isolates failures—a sudden rate limit on Twilio SMS does not slow down email ticket delivery or push alerts.
- **Token Bucket Rate Limiting:** Outbound calls to external gateways enforce token-bucket rate limiters configured to provider tier quotas (e.g., 50 SMS/second on Twilio).
- **DLQ Reprocessing:** Operations engineers monitor notifications-dlq lag via Grafana alerts, triggering replay or automated reprocessing once partner gateway incidents are resolved.